The problems come when multiple threads call `push()` concurrently or multiple threads call `pop()` concurrently. Let's look at `push()` first. If you have two threads calling `push()` concurrently, they both allocate new nodes to be the new dummy node ❸, both read the *same* value for `tail` ❹, and consequently both update the data members of the same node when setting the `data` and `next` pointers ❺, ❻. This is a data race!

There are similar problems in `pop_head()`. If two threads call concurrently, they will both read the same value of `head`, and both then overwrite the old value with the same `next` pointer. Both threads will now think they've retrieved the same node—a recipe for disaster. Not only do you have to ensure that only one thread `pop()`s a given item, but you also need to ensure that other threads can safely access the `next` member of the node they read from `head`. This is exactly the problem you saw with `pop()` for your lock-free stack, so any of the solutions for that could be used here.

So if `pop()` is a "solved problem," what about `push()`? The problem here is that in order to get the required happens-before relationship between `push()` and `pop()`, you need to set the data items on the dummy node before you update `tail`. But this then means that concurrent calls to `push()` are racing over those very same data items, because they've read the same `tail` pointer.

Handling Multiple Threads in Push()

One option is to add a dummy node between the real nodes. This way, the only part of the current `tail` node that needs updating is the `next` pointer, which could therefore be made atomic. If a thread manages to successfully change the `next` pointer from `nullptr` to its new node, then it has successfully added the pointer; otherwise, it would have to start again and reread the `tail`. This would then require a minor change to `pop()` in order to discard nodes with a null data pointer and loop again. The downside here is that every `pop()` call will typically have to remove two nodes, and there are twice as many memory allocations.